Changia

Blockchain-Based NGO Donation Management System

Every shilling provable, from mobile money to the block.

Candidate **Nasibu Y. Isaka** · 2106307225519

Supervisor Dr Gustaph Sanga

Module Project Realization · COU 08204 · BENG22COE

DIT · Department of Computer Studies

MINI 2 · CHAPTERS 5 TO 7

Dar es Salaam Institute of Technology

Department of Computer Studies · NTA Level 8

LIVE SYSTEM

changia-hazel.vercel.app

Ethereum Sepolia

`0x353b6cdaD14774412B5C...`

[Presentation date]

What this presentation covers

Three chapters, one working system. Everything shown is running in production unless explicitly labelled as design.

05

System Requirement Specification

Existing system analysis, the proposed system, functional and non-functional requirements, user and system requirements, feasibility, and the SRS use-case model.

06

System Design

Architecture, data and process modeling, database design, UML models, interface design, the planned fraud-detection model, algorithms and security design.

07

Implementation, Testing and Results

Development environment, module implementation, testing strategy, test cases, measured results, discussion, and the live system itself.

Chapters 1 to 4 and Chapter 8 sit outside MINI 2. This session covers the requirement specification, the design, and the built and tested system.

The system, in one slide

A donation platform where every completed transaction leaves a permanent, publicly checkable record.

14
database tables

24
functional requirements

8
delivery stages

203
automated checks passed

WHAT MAKES IT DIFFERENT

Donors pay with mobile money, the way they already do. The platform hashes each completed donation and writes that proof to a smart contract on a public Ethereum network.

The donor never touches a wallet, never holds cryptocurrency, and never sees a block. They get a receipt number. Anyone holding that number can verify the donation independently, without an account and without asking us.

DEPLOYED AND REACHABLE

changia-hazel.vercel.app
Frontend, Vercel

changia-api.onrender.com
API, Render · PostgreSQL, Neon

Ethereum Sepolia
`0x353b6cdaD14774412B5C7612F0C0D153378F213A`

CHAPTER 5

System Requirement Specification

5

5.1 Introduction

What this chapter establishes before any design work begins.

- This chapter defines WHAT the system must do, stated precisely enough that every requirement can later be traced to a test case.
- The problem is trust. Tanzanian donors give through mobile money and receive a payment confirmation, but no independently verifiable record of what happened to the money afterwards.
- Stakeholders are donors, community fundraisers, beneficiaries, platform administrators, and the regulators who oversee charitable giving.

READING ORDER

Existing system, proposed system, functional and non-functional requirements, user and system requirements, feasibility, then the SRS model.

5.2 Existing System Analysis

Three families of system exist today. Each fails a different part of the trust problem.

Manual NGO records

- Spreadsheets and paper receipts
- Records editable after the fact
- No donor-facing visibility at all
- Reconciliation is slow and manual

Direct mobile money

- Familiar and widely adopted
- Confirms payment, not utilisation
- No campaign or beneficiary context
- No public audit trail

Crypto donation platforms

- Genuinely immutable records
- Require cryptocurrency to donate
- Unusable for the ordinary Tanzanian donor
- Wallet and volatility barriers

WEAKNESSES CARRIED FORWARD TO 5.3

- W1 Records can be altered after the fact, so a donation history cannot be trusted as evidence.
- W2 Donors see a payment confirmation, never proof of what the funds were used for.
- W3 Transparent alternatives demand cryptocurrency, excluding the users who actually give.
- W4 Nothing separates the person who benefits from a payout from the person who releases it.

5.3 Proposed System

Changia. Familiar payment in front, immutable proof behind, and separation of duties on every payout.

W1

Immutable records

Every donation writes a proof hash to a smart contract on Ethereum Sepolia. The database row can be read, never silently rewritten.

W2

Public verification

Any person can enter a receipt number on a public page and see the campaign, amount, date and transaction hash. No login required.

W3

Local payment methods

Donors pay with mobile money or bank transfer. Blockchain stays invisible to them. No wallet, no cryptocurrency, no volatility.

W4

Separation of duties

Beneficiary verification is administrator-only, and payouts above a cumulative threshold need a second administrator who is never the initiator.

IN SCOPE

- Web platform for donors, fundraisers and administrators
- Mobile money and bank donations in Tanzanian Shillings
- On-chain proof for donations and disbursements
- Public receipt verification
- Reporting, audit logging and notifications

OUT OF SCOPE

- Cryptocurrency as a payment method
- Cryptocurrency trading or custody
- A trained fraud-detection model in this phase, designed in 6.9
- Native mobile applications

5.4 Functional Requirements

Written as short, testable statements and numbered so each traces to a test case in 7.6.

DONOR

FR-01	The system shall allow a visitor to register as a donor using an email address or a username.
FR-02	The system shall authenticate a user and lock the account after five consecutive failed attempts.
FR-03	The system shall allow any visitor to browse, search, filter and sort active campaigns.
FR-04	The system shall allow an authenticated donor to donate to an active campaign by mobile money or bank transfer.
FR-05	The system shall generate a downloadable PDF receipt for every completed donation.
FR-06	The system shall record an immutable blockchain proof for every completed donation.
FR-07	The system shall present a donor with their donation history and monthly summary.
FR-08	The system shall award Impact Points to a donor through an append-only ledger.

FUNDRAISER

FR-09	The system shall allow a donor to apply to become a fundraiser, or to register as one directly.
FR-10	The system shall allow a fundraiser to create and edit the campaigns they own.
FR-11	The system shall hold every fundraiser campaign in Pending Review until an administrator approves it.
FR-12	The system shall allow a fundraiser to add beneficiaries to their own campaigns.
FR-13	The system shall allow a fundraiser to release payouts to verified beneficiaries while cumulative self-released funds stay below the threshold.

5.4 Functional Requirements

Continued. Administrator and system-level requirements.

ADMINISTRATOR

FR-14	The system shall allow an administrator to approve or reject a fundraiser application, with a reason.
FR-15	The system shall allow an administrator to review, approve or reject campaigns awaiting review.
FR-16	The system shall restrict beneficiary verification to administrators, and pay only verified beneficiaries.
FR-17	The system shall require a second administrator for any disbursement at or above the threshold, and block the initiator from approving it.
FR-18	The system shall allow an administrator to suspend or deactivate a user, revoking every active session.
FR-19	The system shall record every administrative write and every login attempt in a read-only audit log.
FR-20	The system shall generate five report types, each exportable as CSV, Excel and PDF.

SYSTEM

FR-21	The system shall verify every payment callback with the provider before creating a donation, and process duplicate callbacks idempotently.
FR-22	The system shall expose public verification by receipt number without revealing donor identity.
FR-23	The system shall notify users in-app and by email on donations, campaign milestones, beneficiary updates and disbursement outcomes.
FR-24	The system shall never write personal data to the blockchain.

Traceability. All 24 requirements are implemented. Section 7.4 maps each module to the requirements it fulfils, and 7.6 maps requirements to executed test cases.

5.5 Non-Functional Requirements

How well the system must perform. Each carries a measurable target rather than an aspiration.

Sec

Security

15-minute access tokens, rotating refresh cookies, bcrypt hashing, lockout after 5 failed attempts, rate limiting on auth routes.

Perf

Performance

Read endpoints respond under 300 ms on a warm instance. The blockchain write is never awaited by the payment callback.

Rel

Reliability

Donation finalisation is one atomic transaction. Duplicate callbacks never double-credit. A failed chain write never loses a donation.

Maint

Maintainability

Strict layering, components under 200 lines, functions under 50 lines, zero lint errors across client and server.

Avail

Availability

Managed hosting with a health endpoint. The system degrades gracefully when the chain client is unconfigured.

Scal

Scalability

Stateless API, pooled managed PostgreSQL, every list endpoint paginated and indexed.

Use

Usability

WCAG AA contrast, full keyboard navigation, responsive from 360 px, light and dark themes.

MEASURED

Targets above are the ones verified in Chapter 7, not estimates written before the build.

5.6 User Requirements

What each class of user needs from the system, in their own terms.

Donor

- Give without creating a wallet
- Receive a receipt that proves the gift
- Verify a donation independently, later
- Track campaigns they have supported
- See recognition for sustained giving

Fundraiser

- Raise funds without registering an NGO
- Run a campaign under clear review rules
- Register beneficiaries for payout
- Release modest payouts without waiting
- Report on their own campaigns only

Administrator

- Vet fundraisers and campaigns before they go live
- Verify that a beneficiary is a real payee
- Approve payouts above the threshold
- Audit every action taken on the platform
- Export compliance reports on demand

Role progression. A user starts as a donor, may become a fundraiser by application or at sign-up, and is only ever made an administrator by another administrator. Administrator is never self-assignable.

5.7 System Requirements

What the system needs to run, and what it needs to prove itself.

Hardware

- Development: Intel i5 or equivalent, 8 GB RAM, SSD
- Client: any device with a modern browser, from 360 px wide
- Server: managed instances, no owned hardware

Software

- Node.js 24, TypeScript
- React 19, Vite, Tailwind CSS v4
- PostgreSQL 16 with Drizzle ORM
- Solidity, Hardhat, Ethers.js

Network

- HTTPS on every origin
- Payment provider callback endpoint
- JSON-RPC access to a Sepolia node
- SMTP relay for transactional email

Blockchain

- Ethereum Sepolia testnet
- TransparencyRegistry
- Owner-only contract writes
- Funded backend wallet for gas
- One transaction per proof

Model and Algorithms

- SHA-256 deterministic proof hash
- Cumulative threshold payout algorithm
- Fraud-detection model, designed in 6.9
- Implementation scheduled, not built

Model and algorithms. The fraud-detection model is specified in section 6.9 and is designed, not implemented. It is stated here because the system requirement exists, not because a model runs today.

5.8 Feasibility Study

Five dimensions. The project is already deployed, which settles most of them empirically.

Technical

- Stack already proven in the working build
- Testnet removes cost and key-custody risk
- Skills held by the development team

Economic

- Managed free tiers for hosting and database
- Sepolia gas paid in faucet test ETH
- Near-zero recurring cost at project scale

Operational

- Mirrors how donors already pay
- Administrator training is minimal
- Fits existing NGO oversight workflows

Legal

- No personal data written on-chain
- Open-source licences throughout
- Audit trail supports compliance reporting

Schedule

- Delivered in staged increments
- Each stage verified before the next
- Deployed and publicly reachable

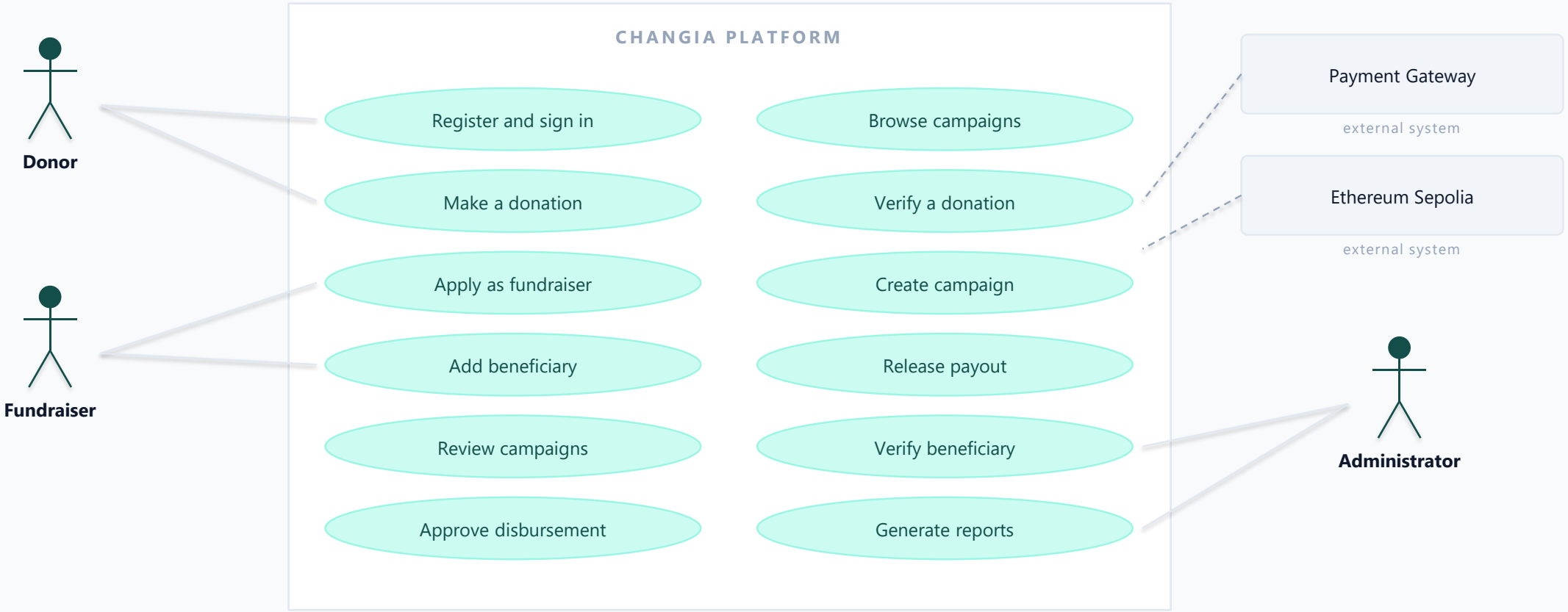
TZS 0
recurring hosting cost at project scale

Faucet ETH
pays all Sepolia gas, no real funds at risk

Live
deployed and publicly reachable today

5.9 SRS · Use-Case Model

Four actors. Two of them are other systems.



CHAPTER 6

System Design

6

Changia

6.1 Introduction

Chapter 5 defined WHAT. This chapter defines HOW.

- The system follows a layered client-server architecture with a strict backend flow of routes, controllers, services and repositories. No layer is ever skipped.
- Three notations carry the design: data flow diagrams for process decomposition, an entity relationship diagram for the data model, and UML for behaviour.
- One architectural rule shapes everything: PostgreSQL holds operational data, the blockchain holds proof only, and the frontend never talks to the chain directly.

FRAMING

Chapter 5 is the contract. Chapter 6 is the mechanism that satisfies it. Every design choice below answers a numbered requirement.

6.2 System Architecture

Layered client-server. The backend never skips a layer, and the frontend never talks to the chain.

Presentation Layer

React 19, TypeScript, Vite, Tailwind v4

Public site, donor account, fundraiser console, administrator console. Talks only to the API, over an Axios service layer.

Application Layer

Node.js, Express, TypeScript

Routes to controllers to services to repositories. Business rules live in services. Owns payments, blockchain writes and authorisation.

Data Layer

PostgreSQL on Neon, Drizzle ORM

14 tables, 11 migrations. Donations are immutable. Foreign keys enforced, every query parameterised.

EXTERNAL SYSTEMS

Payment Provider

Adapter interface. Mock and AzamPay behind one seam.

Ethereum Sepolia

TransparencyRegistry.sol. Proof records only.

THE RULE

PostgreSQL holds operational data. The chain holds proof only. Personal data never leaves the database.

6.2.1 Blockchain Design

What goes on-chain, what never does, and why the split matters.

ON-CHAIN

- Record type, donation or disbursement
- SHA-256 proof hash
- Amount and timestamp
- Internal reference identifier

NEVER ON-CHAIN

- Donor name, email or phone number
- Beneficiary identity or contact details
- Payment reference or provider payload
- Anything that could identify a person

Donation commits
`in PostgreSQL`

→

Hash the payload
`SHA-256, deterministic`

→

Submit to contract
`registerDonation()`

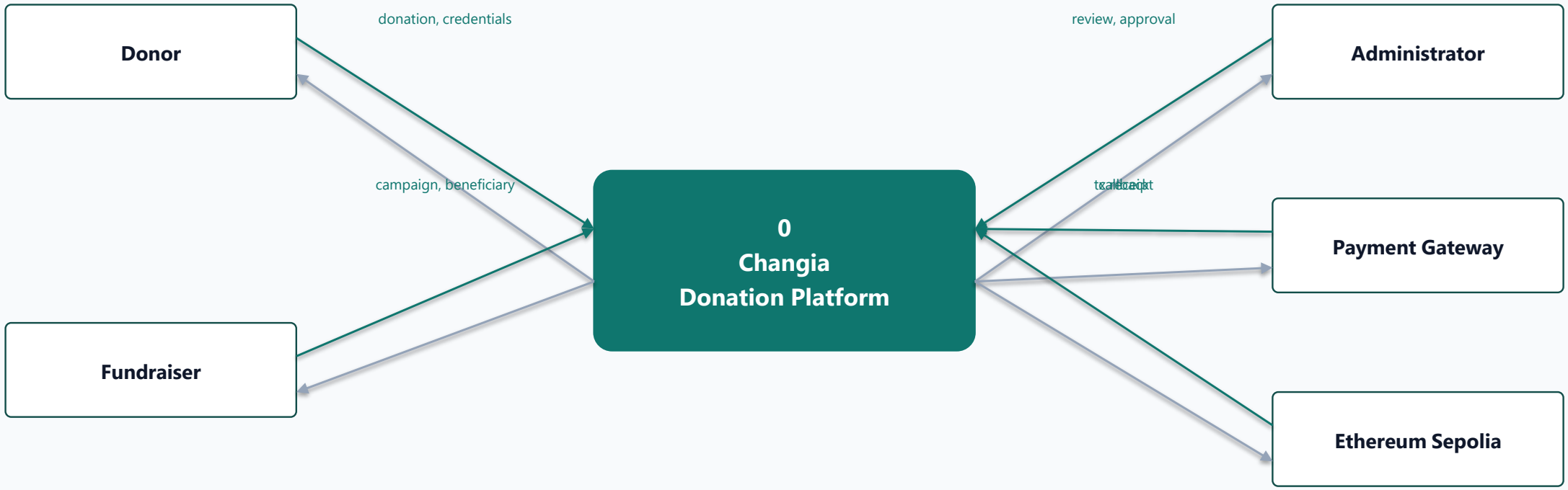
→

Store the receipt
`tx hash, block, network`

Why a hash and not the data. A hash proves the record has not changed without publishing anything about the person who gave. Verification compares a recomputed hash against the chain.

6.3 Data Modeling · Context Diagram

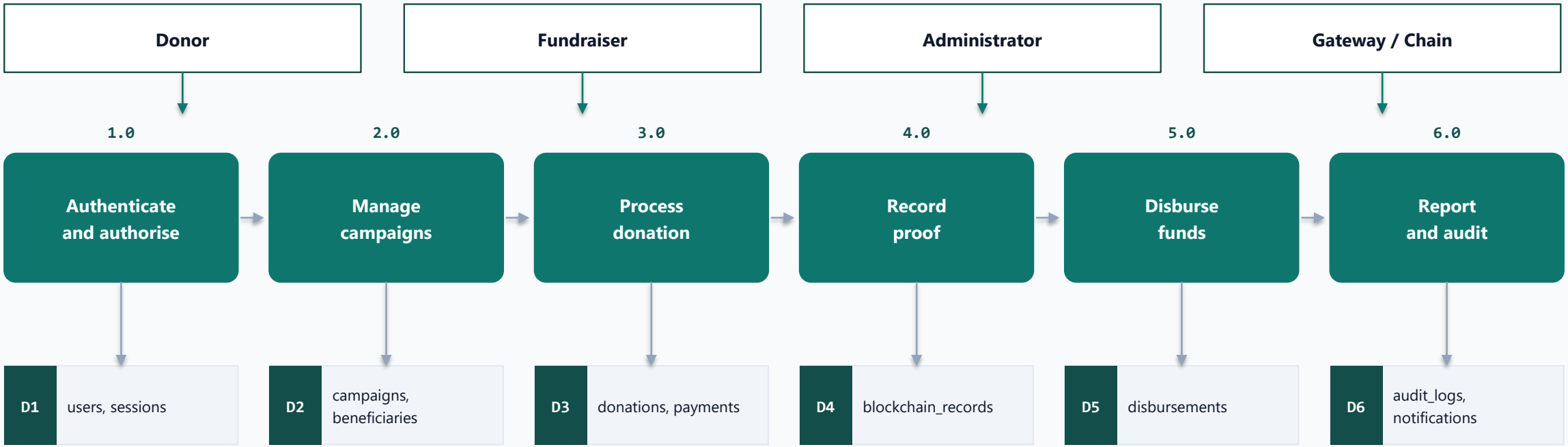
Level 0. The system as a single process against the entities it exchanges data with.



Teal arrows carry data into the system, grey arrows carry it back out. Level 1 on the next slide decomposes process 0 into its six major sub-processes.

6.3.1 Data Flow Diagram · Level 1

Process 0 decomposed. Six sub-processes and the stores they read and write.



Read the ordering, not just the boxes. Money is written to D3 and D5 before any proof is written to D4. The proof is always downstream of the transaction it describes, which is why a failed chain write can never destroy a donation record.

6.4 Database Design

14 tables, 11 applied migrations, managed PostgreSQL on Neon.

Identity

`users`

`sessions`

`password_resets`

`fundraiser_applications`

Fundraising

`campaigns`

`beneficiaries`

Money

`donations`

`payment_transactions`

`disbursements`

`disbursement_approvals`

Proof and oversight

`blockchain_records`

`audit_logs`

`notifications`

`reward_events`

DESIGN RULES THAT CONSTRAIN THE SCHEMA

- Donations are never updated or deleted. Correction happens by a new row, never a rewrite.
- blockchain_records is polymorphic: one table proves both donations and disbursements, mirroring the contract's own record type.
- reward_events is an append-only ledger. A donor's balance is the sum of history, never a stored figure.
- Users are soft-deleted so that donation history keeps its owner.

14
tables

11
migrations

6.4.1 Entity Relationship Diagram

Core entities. The full 14-table model is in the appendix.

blockchain_records is polymorphic: record_type plus reference_id point at either a donation or a disbursement, mirroring the contract's own record type.

6.5 Class Diagram · Service Layer

Business logic lives in services. Controllers stay thin, repositories stay dumb.

AuthService

- userRepo
- sessionRepo

+ register()
+ login()
+ rotateRefresh()

DonationService

- donationRepo
- campaignRepo

+ finalise()
+ history()
+ receipt()

PaymentService

- provider: PaymentProvider

+ createSession()
+ handleCallback()

BlockchainService

- contract: Contract

+ recordDonation()
+ reconcile()

DisbursementService

- disbursementRepo

+ initiate()
+ approve()
+ threshold()

ReportService

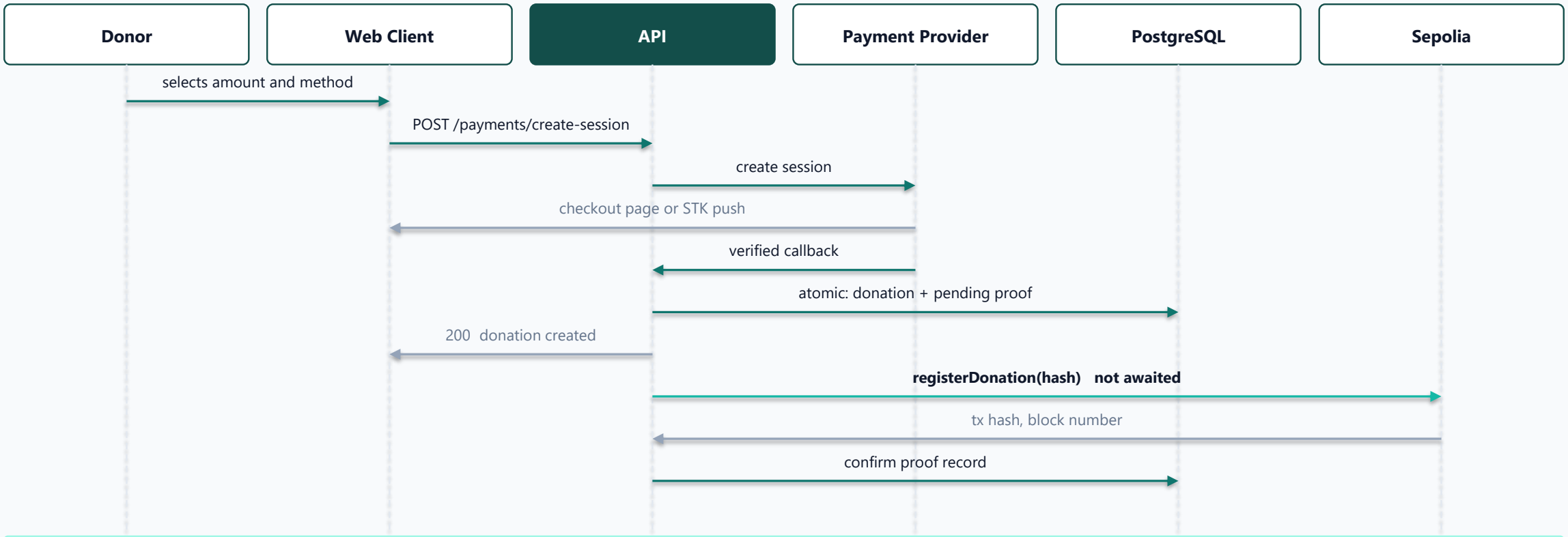
- exporters

+ build()
+ export()

PaymentProvider is an interface, not a class. The mock provider and the AzamPay adapter both satisfy it, so swapping providers never touches DonationService.

6.5.1 Sequence Diagram · Donation

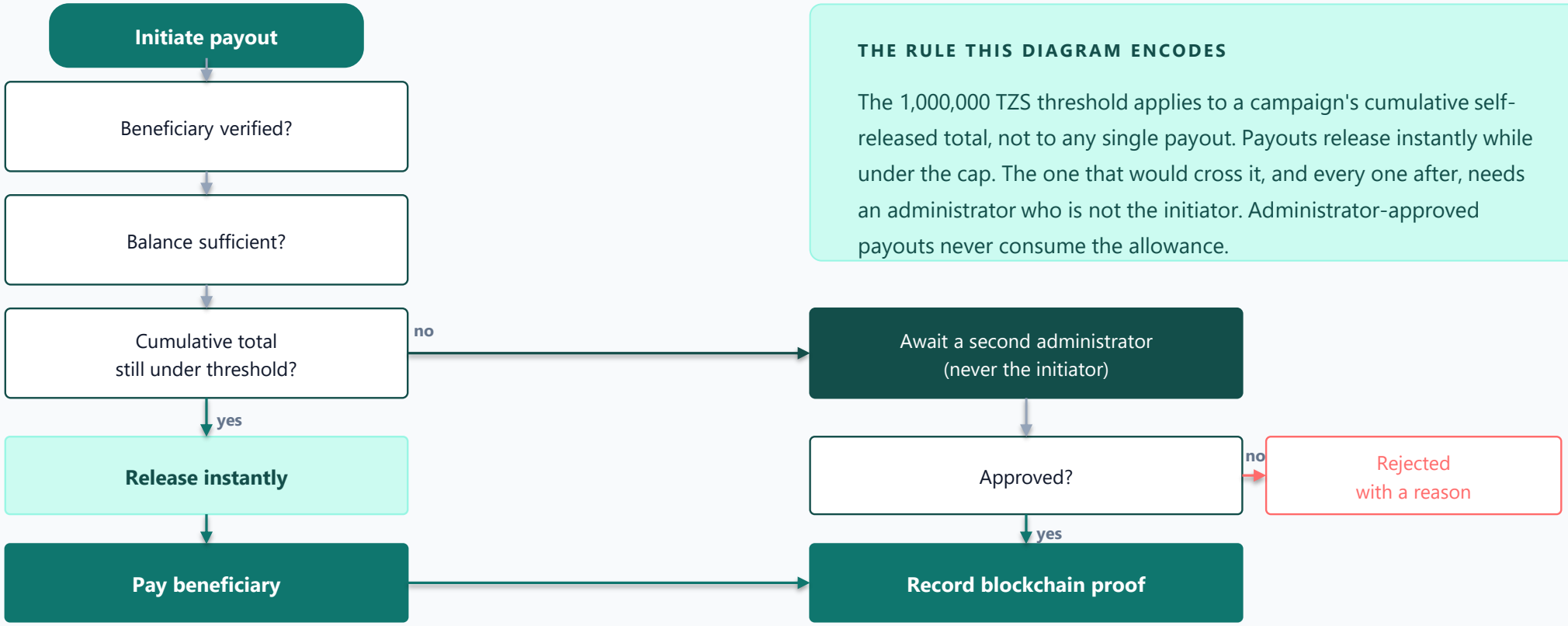
The core transaction, end to end. Note where the chain write sits.



The donor's response does not wait for the blockchain. Step 7 returns as soon as the database commits. If the chain write in step 8 fails, the donation still exists and the proof reconciles on the next verification read. Blockchain failure must never lose donation data.

6.5.2 Activity Diagram · Disbursement Approval

The control that stops any one person from both benefiting and releasing.

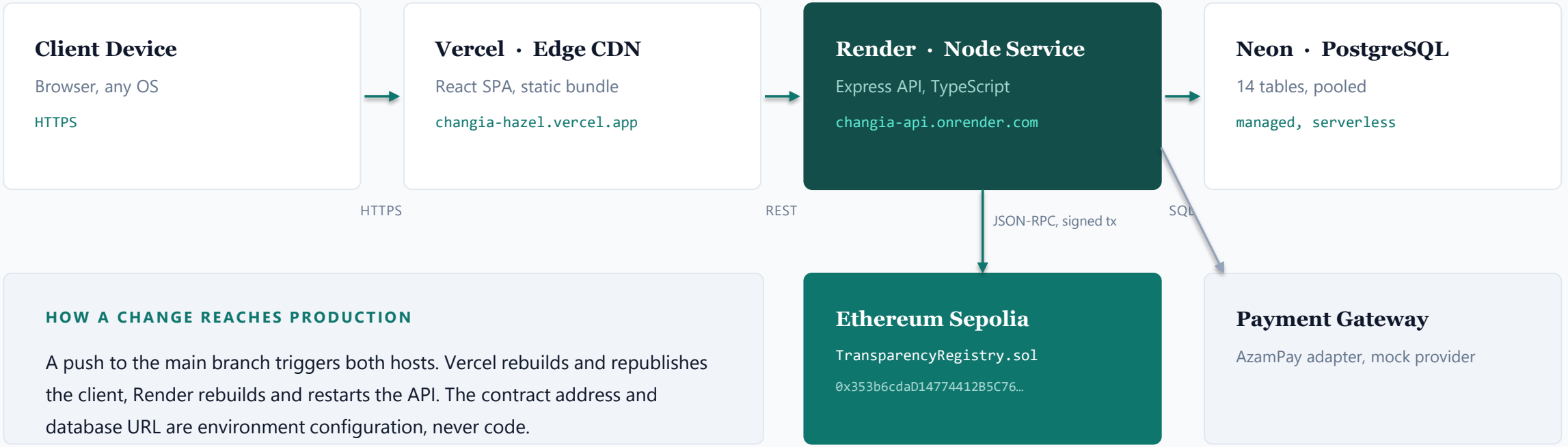


THE RULE THIS DIAGRAM ENCODES

The 1,000,000 TZS threshold applies to a campaign's cumulative self-released total, not to any single payout. Payouts release instantly while under the cap. The one that would cross it, and every one after, needs an administrator who is not the initiator. Administrator-approved payouts never consume the allowance.

6.5.3 Deployment Diagram

Real hosts, real URLs. This is what is running right now.



HOW A CHANGE REACHES PRODUCTION

A push to the main branch triggers both hosts. Vercel rebuilds and republishes the client, Render rebuilds and restarts the API. The contract address and database URL are environment configuration, never code.

6.7 User Interface Design

One design system across four consoles. Institutional, not charitable-cliché.

DESIGN TOKENS

- Harbor Teal #0f766e
- Ink Slate #0f172a
- Slate Mist #64748b
- Teal Tint #ccfbf1

TYPOGRAPHY AND BEHAVIOUR

- Source Serif 4 for display, Inter for working text
- Light and dark themes, no flash on first paint
- Motion respects prefers-reduced-motion throughout
- Every state designed: loading, empty, error, success

Authentication

Sign in, register, recovery

Campaign detail

Progress, donate, verification

Consoles

Donor, fundraiser, administrator

Public verify

Receipt lookup, no login

6.9 Fraud-Detection Model Design

DESIGNED · IMPLEMENTATION SCHEDULED

Dataset, features and model specified in this phase. Implementation is scheduled for a later stage and is not part of the current build.

Objective

- Score donations and payouts for review
- Flag anomalies to an administrator
- Assist a human decision, never replace it

Dataset

- The platform's own operational history
- donations, payment_transactions
- disbursements, audit_logs
- No external data needed

Candidate features

- Amount against campaign mean
- Donation velocity per account
- Beneficiary age at first payout
- Cumulative self-released ratio
- Failed-login proximity in time

Model and split

- Isolation Forest for the unlabelled start
- Gradient-boosted trees once labelled
- Labels from administrator review outcomes
- Chronological 70/15/15 split

WHY THIS IS FEASIBLE

The schema already emits every candidate feature, so no new data capture is needed before training. The audit log supplies ground truth: each flagged item records what the reviewing administrator decided.

SCOPE STATEMENT

No model is trained or running in the current build. This section specifies the design only. Section 7 reports no ML results because there are none to report.

6.10 Algorithms and Flowchart

6.9.1 the proof algorithm, and 6.10 the path a donation takes through the system.

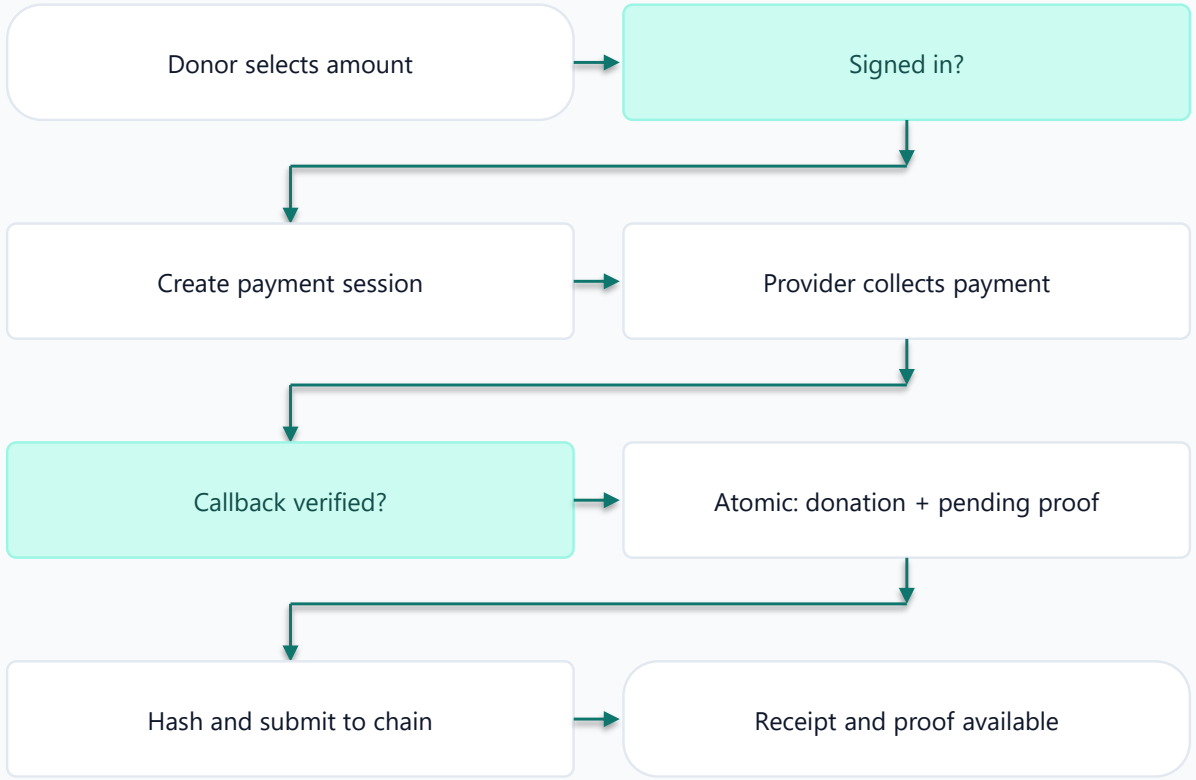
PROOF-HASH ALGORITHM

1. Finalise the donation in one atomic transaction
2. payload = donation id + campaign id + amount
   + receipt number + payment reference
   + timestamp
3. proof = SHA-256(payload)
4. registerDonation(proof) on the contract
5. Store tx hash, block number and network
6. On failure, leave pending and self-heal on read

DETERMINISTIC BY DESIGN

The same donation always produces the same hash, so anyone can recompute it and compare against the chain.

FLOWCHART OF THE WORKING SYSTEM



6.11 Security Design

Each control below answers the Security non-functional requirement in 5.5.

Authentication

Short-lived JWT access token, rotating httpOnly refresh cookie, bcrypt password hashing, lockout after five failed attempts.

Authorisation

Role-based access control derived from the role enum. Ownership checks on every fundraiser resource.

Data protection

HTTPS everywhere, parameterised queries throughout, no personal data written on-chain, uploads renamed server-side.

Input validation

Every request body validated before it reaches a service. Helmet, CORS allowlist and rate limiting at the edge.

Accountability

Read-only audit log of every administrative write and every login attempt, searchable and filterable.

Separation of duties

Beneficiary verification is administrator-only. A payout initiator can never approve their own disbursement.

Traceability. NFR Security in 5.5 specified 15-minute access tokens, rotating refresh, bcrypt and lockout after five attempts. All four are implemented and verified by test cases TC-01 and TC-02.

CHAPTER 7

Implementation, Testing and Results

7

Changia

7.1 Introduction

The system was built in staged increments, each verified before the next began.

8	203	2	1
delivery stages completed	automated checks passed	non-passes, both explained	real bug found and fixed

Chapter 6 described a design. This chapter shows that it was built, that it was tested, and that it runs.

Every figure on the following slides comes from an executed test run against a live database and a real blockchain, not from an estimate.

THE SHORT VERSION

It is deployed, it is public, and you can verify a donation on your own phone before the end of this presentation.

7.2 · 7.3 Development Environment and Software Tools

One language across the whole stack, which keeps the contracts, the API and the client consistent.

Development Environment

- Windows 11, Visual Studio Code
- Git and GitHub, main and develop branches
- Node.js 24, npm workspaces
- Local Hardhat node for chain development
- Neon branch database for integration

Software Tools

- TypeScript across client, server and contracts
- Vite, Tailwind CSS v4, shadcn/ui
- Drizzle ORM and Drizzle Kit migrations
- Hardhat, Ethers.js, Solidity
- ESLint, pdfkit, exceljs, Recharts

ENGINEERING DISCIPLINE

Conventional commits
small, reviewable history

Lint and build gate
zero errors before any commit

Staged verification
each stage E2E-tested before the next

Reproducible deck
this presentation is generated by script

7.4 Implementation of Modules

Eight modules. Each maps to the functional requirements it fulfils.

Authentication

`FR-01, FR-02, FR-18`

Rotating refresh tokens, RBAC middleware, account lockout.

Campaigns

`FR-03, FR-10, FR-11, FR-15`

Ownership model with a six-state lifecycle and a review gate.

Donations and Payments

`FR-04, FR-05, FR-21`

Provider abstraction, idempotent callbacks, real PDF receipts.

Blockchain

`FR-06, FR-22, FR-24`

Deterministic proof hash, background write, live reconciliation.

Beneficiaries

`FR-12, FR-16`

Administrator-only verification gates every payout.

Disbursements

`FR-13, FR-17`

Cumulative threshold, dual approval, self-approval blocked.

Rewards

`FR-08`

Append-only Impact Points ledger with derived balance and tier.

Reporting and Audit

`FR-19, FR-20`

Five report types, three export formats, one shared export utility.

7.4.1 Two Problems Worth Describing

Both are correctness problems that only appear under failure or duplication.

Problem 1

A failed blockchain write must never lose a donation

The chain write is fired after the donation commits and is never awaited by the payment callback. If it crashes, the proof stays pending and the next verification read reconciles it live against the contract.

Problem 2

A payment provider may deliver the same callback twice

Finalisation runs in one atomic transaction keyed on the payment reference. A duplicate callback resolves to the same donation, and the campaign total is credited exactly once.

THE PRINCIPLE BEHIND BOTH

Money is the source of truth, proof is a consequence of it. The database transaction is the thing that must never fail or double-apply. Everything downstream, including the blockchain write, is allowed to retry, lag, or self-heal without ever putting a donation at risk.

7.5 Testing Strategy

Four levels. Each answers a different question about the system.

Unit

Unit Testing

Hardhat tests over the smart contract: registration, duplicate rejection, owner-only writes, retrieval.

4 / 4 passing

Int

Integration Testing

Automated API suites per stage against live PostgreSQL and a real chain, asserting cross-module behaviour.

Per stage, below

Sys

System Testing

Real-browser checks of complete journeys: register, donate, verify, administer, review.

28 browser checks

UAT

User Acceptance Testing

Structured acceptance walkthrough on the live deployment against the documented acceptance criteria.

Developer-executed

WHAT WAS TESTED AGAINST

Not mocks. Integration suites ran against the live Neon PostgreSQL instance and a real blockchain node, and one check cross-referenced the node's own log to confirm the exact transaction hash the API returned.

7.6 Test Cases

A representative sample. Each maps back to a numbered requirement from 5.4.

ID	Description	Input	Expected result	Status
TC-01	Donor registration	Valid email and password	Account created with donor role	Pass
TC-02	Account lockout	5 consecutive wrong passwords	Account locked, further attempts refused	Pass
TC-03	Donation end to end	Amount, verified callback	Donation, receipt and pending proof created	Pass
TC-04	Duplicate callback	Same payment reference twice	One donation, campaign credited once	Pass
TC-05	On-chain proof	Completed donation	Tx hash matches the node's own log	Pass
TC-06	Public verification	Receipt number, no login	Campaign, amount, date and hash, no donor identity	Pass
TC-07	Self-approval block	Initiator approves own payout	Rejected, second administrator required	Pass
TC-08	Cross-account isolation	Donor reads another's donation	403, no data disclosed	Pass

TC-05 is the one to notice. The test does not trust the API's own response. It reads the blockchain node's log independently and asserts the two transaction hashes match.

7.7 · 7.8 Results and Discussion

Measured outcomes across every stage, and an honest account of what did not pass.

VERIFICATION RESULTS BY STAGE

Authentication	18 / 18	browser
Donations	19 / 19	API
Blockchain	14 / 14	API
Blockchain	10 / 10	browser
Administrator	46 / 47	API
Fundraisers	40 / 40	API
Regression	52 / 53	API
Smart contract	4 / 4	Hardhat

REQUIREMENTS MET

- All 24 functional requirements are implemented and exercised by an automated check.
- Non-functional targets hold: reads land under 300 ms warm, and the chain write never blocks a donation response.
- Every completed donation and disbursement carries a real transaction hash on a public network.

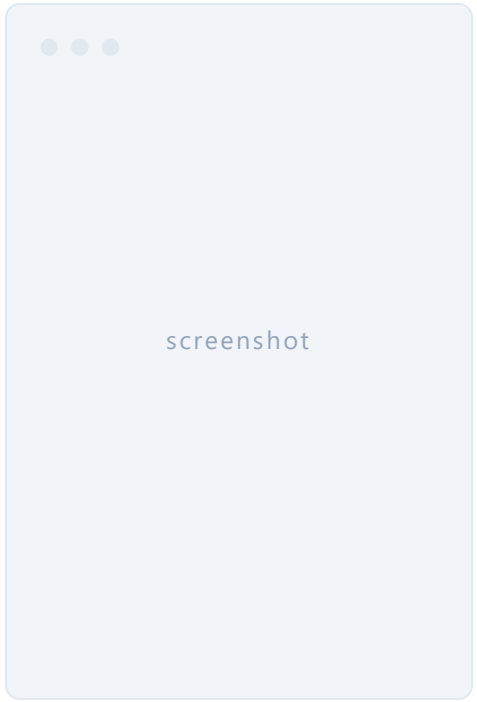
WHAT DID NOT PASS, AND WHY

- Two checks did not pass, neither a functional defect: one asserted a status that had already advanced asynchronously, one was a transient database cold-start timeout.
- One real bug was found and fixed: a failing notification insert could crash the server through an unhandled rejection. It now fails internally, matching the fire-and-forget pattern used elsewhere.
- User acceptance testing is developer-executed. Independent end-user acceptance is not yet run.

Overall: 203 of 205 automated checks pass. Neither failure is a functional defect.

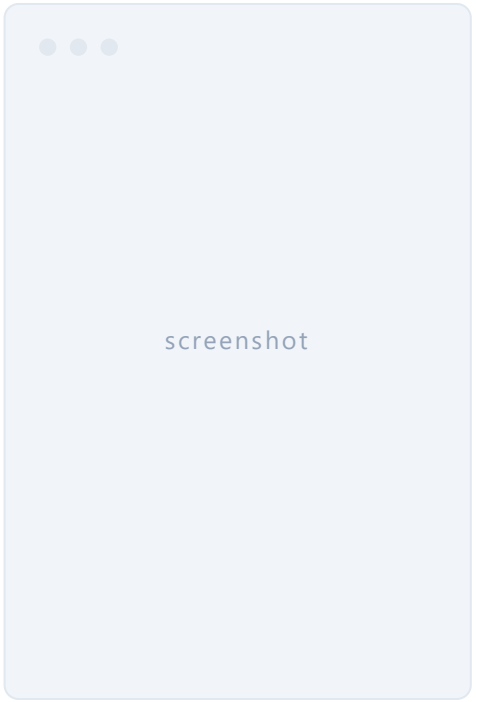
7.9 System Screenshots

One journey, end to end. Public entry through to administrative oversight.



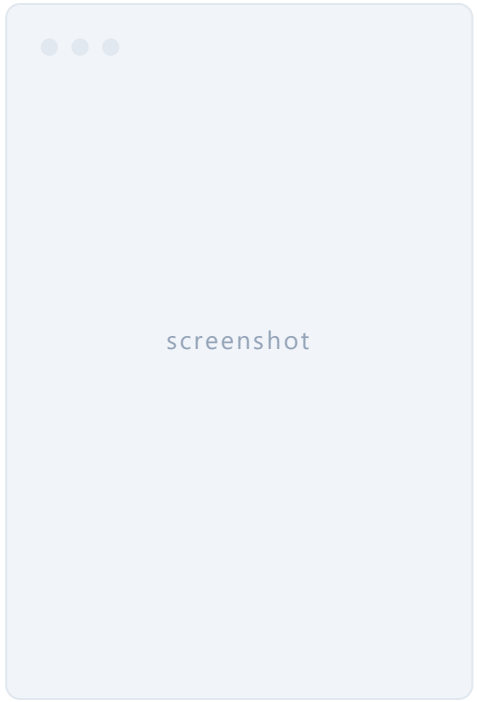
Landing and campaigns

Public entry, live campaign data



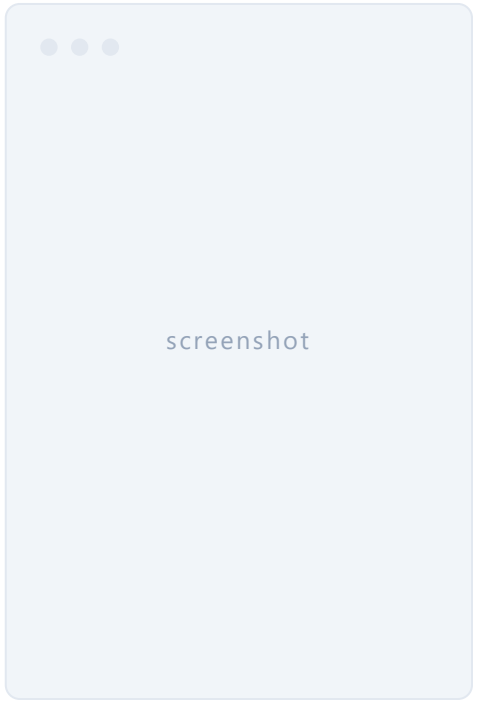
Donation and checkout

Amount, method, provider session



Receipt and verification

PDF receipt, on-chain proof state



Administrator console

Dashboard, reviews, disbursements

LIVE DEMONSTRATION

Verify a donation yourself, right now.

Scan the code. Open the platform. Enter any receipt number on the public verify page.

No account is required and no permission is needed from us. That is the entire point of the system: the proof does not depend on trusting the platform that produced it.

Platform	**changia-hazel.vercel.app**
Network	Ethereum Sepolia
Contract	`0x353b6cdaD14774412B5C7612F0C0D153378F213A`

Scan to open Changia

changia-hazel.vercel.app

7.8.1 Limitations and Future Work

Stated plainly. Every one of these is a known boundary of the current phase, not a surprise.

CURRENT LIMITATIONS

Sepolia, not mainnet

Proofs are real and publicly verifiable, but carry no monetary value. Migration is a configuration change, not a rewrite.

Mobile money runs on the sandbox provider

The AzamPay adapter is built behind the payment seam. Live credentials are pending, so the mock provider serves the demo.

Free-tier cold starts

The API sleeps when idle, so the first request after a quiet period can take up to a minute.

Uploads are on ephemeral disk

Campaign images do not survive a redeploy. Object storage is the fix.

Fraud detection is designed, not trained

Section 6.9 specifies the dataset, features and model. No model runs today.

FUTURE WORK

- Implement the fraud-detection module specified in 6.9, starting with the unlabelled Isolation Forest baseline
- Complete AzamPay production integration once merchant credentials are issued
- Move object storage off ephemeral disk
- Evaluate a low-cost L2 or mainnet for production proofs
- Swahili localisation across the donor-facing surface
- Independent user acceptance testing with real donors and NGO staff

NEXT STAGE

The fraud-detection module designed in 6.9 is the immediate next piece of work. The schema already produces every feature it needs.

Thank you

Questions and discussion

CHANGIA

changia-hazel.vercel.app

Nasibu Y. Isaka

2106307225519

Supervised by Dr Gustaph Sanga